Discrete Applied Mathematics 7 (1984) 115-130
North-Holland

DOMINATION, INDEPENDENT DOMINATION, AND DUALITY IN STRONGLY CHORDAL GRAPHS*

Martin FARBER
Department of Combinatorics and Optimization, University of Waterloo, Waterloo, Ontario, Canada N2L 3G1

Received 20 January 1982
Revised 24 November 1982

We present polynomial algorithms to locate minimum weight dominating sets and independent dominating sets in strongly chordal graphs. We utilize an intimate relationship between strongly chordal graphs and totally balanced matrices to show that the domatic number achieves its theoretical lower bound in strongly chordal graphs and to efficiently solve certain optimization problems for totally balanced matrices.

1. Introduction

A set S of vertices of a graph $G=(V,E)$ is *dominating* if every vertex in $V-S$ is adjacent to some vertex in S, and is *independent* if no two vertices in S are adjacent. An *independent dominating set* in G is a set of vertices of G which is both independent and dominating. If the vertices of G are assigned real weights then the weight of S is the sum of the weights of its elements. The *domatic number* of G, denoted $d(G)$, is the maximum number of pairwise disjoint dominating sets in G.

The problems of locating minimum cardinality dominating sets, minimum weight dominating sets, and minium cardinality independent dominating sets have been studied previously. Since each of these problems is NP-hard for arbitrary graphs [14], most of the work has been devoted to solving these problems for certain classes of graphs. A linear algorithm for locating a minimum cardinality independent dominating set in a tree is given in [3], and we have recently solved the problem for chordal graphs [11]; see also [10]. Surprisingly, the problem of locating a minimum weight independent dominating set in a chordal graph with integer vertex weights is NP-hard [5]. The problem of locating a minimum cardinality dominating set in a chordal graph is also NP-hard [4]. However, efficient algorithms have been found to solve this problem for several subclasses of chordal graphs. In particular, linear algorithms to solve this problem have been designed for trees [7], powers of trees

* The material in Sections 2, 3, and 4 is taken from the author's Ph.D. thesis which was written under the supervision of Dr. Pavol Hell and issued as Technical Report 81-13, Computing Science Department, Simon Fraser University.

0166-218X/84/$3.00 © 1984, Elsevier Science Publishers B.V. (North-Holland)

(when the underlying tree is known) [23], and directed path graphs (given an appropriate path representation) [4]. (We note that interval graphs are directed path graphs.) In addition, linear algorithms to locate minimum weight dominating sets in trees are given in [9] and [19].

The class of strongly chordal graphs, introduced in [10], is an interesting subclass of chordal graphs which properly includes trees, powers of trees, and directed path graphs. We present efficient algorithms to locate minimum weight dominating sets (Section 3) and independent dominating sets (Section 4) in these graphs. These algorithms are linear, given a certain ordering on the vertices (see Section 2). In addition, we utilize an intimate relationship between strongly chordal graphs and totally balanced matrices to show that the domatic number of a strongly chordal graph achieves its theoretical lower bound (Section 5), and to efficiently solve certain optimization problems for totally balanced matrices (Section 6).

We remark that we have designed a polynomial algorithm to recognize a strongly chordal graph which constructs the required ordering on the vertices, if one exists [12]; see also [10]. Moreover, the ordering can be found in linear time for trees, powers of trees (when the underlying tree is known), and directed path graphs (given an appropriate path representation) [10].

We employ the same general method in each of the algorithms presented here and in [9] and [11]. This method differs from the methods used in [3], [4], [7], [19], and [23], which relied upon ad hoc calculations. We begin by putting the problem at hand into the framework of linear programming. In particular, we exhibit a linear program with 0–1 solutions which correspond to potential solutions to the problem at hand. Each algorithm has two stages. The first stage is a greedy algorithm which solves the dual of the linear program mentioned above, and the second stage uses this solution to the dual to read off a feasible 0–1 solution to the primal. By verifying that these solutions satisfy the conditions of complementary slackness, we are able to show that they are optimal, and hence yield a solution to the problem at hand. (We refer the reader to any standard textbook on linear programming, e.g. Simmons [22], for an introduction to linear programming duality and the principle of complementary slackness.) We emphasize that it is because the associated linear programs have 0–1 optimal solutions that the algorithms solve the original problems.

After submitting the original version of this paper for publication, we learned of the related work of Hoffman, Kolen and Sakarovitch [15] and Chang and Nemhauser [6]. Hoffman, Kolen and Sakarovitch study totally balanced matrices and solve a problem which is slightly more general than that treated in Section 6. Chang and Nemhauser define a class of graphs, which they call *sun-free triangulated graphs*, in terms of certain forbidden induced subgraphs, and show that a minimum cardinality dominating set in any power of such a graph can be found in polynomial time. The definition of a sun-free triangulated graph coincides exactly with a forbidden induced subgraph characterization of the class of strongly chordal graphs presented in [10, 12]. Lubiw recently showed that any power of a strongly chordal

graph is strongly chordal [18]. In light of these facts, it becomes clear that the weighted domination algorithm presented in this paper actually solves a more general problem than that solved by Chang and Nemhauser.

2. Strongly chordal graphs

A graph is *chordal* if it contains no cycle of length greater than three as an induced subgraph. Rose [20] gives an interesting characterization of chordal graphs.

Definition [20]. A *perfect elimination ordering* of a graph $G=(V,E)$ is an ordering $v_1, v_2, \ldots, v_n$ of V with the property that, for each i, j and k, if $i<j<k$ and $[v_i, v_j], [v_i, v_k] \in E$, then $[v_j, v_k] \in E$.

Theorem 2.1 [20]. *A graph is chordal if and only if it admits a perfect elimination ordering.*

Using the above characterization and an algorithm by Rose, Tarjan, and Lueker [21] to construct a perfect elimination ordering in linear time, we gave a linear algorithm to locate a minimum cardinality independent dominating set in a chordal graph [11]; see also [10]. Unfortunately, our technique does not lend itself to the problem of locating minimum weight independent dominating sets in chordal graphs because the associated linear program may not have 0–1 optimal solutions. However, if we require that a stronger ordering condition hold (see below), then our method can be employed to solve this problem.

Definition. A *strong elimination ordering* of a graph $G=(V,E)$ is an ordering $v_1, v_2, \ldots, v_n$ of V satisfying the following two conditions for each i, j, k and l:

(a) If $i<j<k$ and $[v_i, v_j], [v_i, v_k] \in E$, then $[v_j, v_k] \in E$.

(b) If $i<j<k<l$ and $[v_i, v_k], [v_i, v_l], [v_j, v_k] \in E$, then $[v_j, v_l] \in E$.

Definition. A graph is *strongly chordal* if it admits a strong elimination ordering.

Notice that an ordering that satisfies condition (a) is a perfect elimination ordering. Hence, every strong elimination ordering is a perfect elimination ordering, and every strongly chordal graph is chordal. In the lemma below we give another description of a strong elimination ordering. First, however, we introduce some notation which will be used throughout the paper.

Notation. Let $G=(V,E)$ be a graph on the vertices $v_1, v_2, \ldots, v_n$. We will write

(i) $i \sim j$ if $i=j$ or $[v_i, v_j] \in E$,

(ii) $i \gtrsim j$ if $i \sim j$ and $i \geq j$, and

(iii) $i \lesssim j$ if $i \sim j$ and $i \leq j$.

Lemma 2.2. *An ordering $v_1, v_2, \ldots, v_n$ of the vertices of G is a strong elimination ordering of G if and only if for each i, j, k, and l with $i \leq j$, $k \leq l$, and $i \sim k$, $i \sim l$, and $j \sim k$ we have $j \sim l$.*

Proof. (Necessity) Suppose $v_1, v_2, \ldots, v_n$ is a strong elimination ordering of G. Suppose, too, that $i \leq j$, $k \leq l$, and $l \sim i \sim k \sim j$. If $i = j$, $k = l$, or $j = l$, then $j \sim l$. Suppose, on the other hand, that $i < j$, $k < l$, and $j \neq l$. By symmetry, we may assume that $i \leq k$. We consider three cases.

Case 1. Suppose $j = k$. Then $i < j < l$ and $[v_i, v_j], [v_i, v_l] \in E$, whence $[v_j, v_l] \in E$, by the definition of a strong elimination ordering.

Case 2. Suppose $j < k$. Then $i < j < k < l$ and $[v_i, v_k], [v_i, v_l], [v_j, v_k] \in E$, whence $[v_j, v_l] \in E$, by the definition of a strong elimination ordering.

Case 3. Suppose $i \leq k < j$. If $i = k$, then $[v_k, v_l] \in E$. Otherwise, $i < k < l$ and $[v_i, v_k], [v_i, v_l] \in E$, whence $[v_k, v_l] \in E$. Consequently, $[v_k, v_l], [v_k, v_j] \in E$ and either $k < l < j$ or $k < j < l$. In either case, $[v_j, v_l] \in E$.

This completes the proof of necessity. The proof of sufficiency is trivial and we omit it. □

In [10] and [12] we give several other characterizations of strongly chordal graphs, including a forbidden induced subgraph characterization.

3. Domination

In this section we will present a linear algorithm to locate a minimum weight dominating set in a strongly chordal graph, given a strong elimination ordering. Throughout this section and the next we will let n denote the number of vertices of a graph and let m denote the number of edges.

Let G be a graph on the vertices $v_1, v_2, \ldots, v_n$ which have been assigned arbitrary real weights $w_1, w_2, \ldots, w_n$. As we mentioned in the introduction, we will locate a minimum weight dominating set in G by solving an associated linear program. Consider the following linear program.

$$
\begin{aligned}
P_1(G): \quad & \text{Minimize} && \sum_{i=1}^{n} w_i x_i, \\
& \text{subject to} && \sum_{i \sim j} x_i \geq 1 \quad \text{for each } j, \\
& && x_i \geq 0 \quad \text{for each } i.
\end{aligned}
$$

By the definition, a set S of vertices of G is a dominating set if and only if, for each j, S contains some vertex v_i such that $i \sim j$. Consequently, there is a one-to-one correspondence between feasible 0–1 solutions to $P_1(G)$ and dominating sets in G. Moreover, an optimal 0–1 solution to $P_1(G)$ corresponds to a minimum weight dominating set in G. Unfortunately, if $w_i < 0$ for any i, then $P_1(G)$ is unbounded

and hence no 0–1 solution to $P_1(G)$ is an optimal solution to $P_1(G)$. Therefore, one cannot simply solve $P_1(G)$ in order to find a minimum weight dominating set in G. Fortunately, this particular obstacle can be overcome.

Let $S, S' \subset V(G)$. We say that S' *dominates* S if every vertex in $S - S'$ is adjacent to some vertex in S'. If S is the singleton $\{v_i\}$ we say that S' dominates v_i. Let $U = \{v_i \in V(G) : w_i \leq 0\}$ and let U^* be the set of all vertices of G which are dominated by U. Let S be a minimum weight set of vertices of $G - U$ which dominates $V(G) - U^*$, where $G - U$ is the subgraph of G induced by $V(G) - U$. Then it is not difficult to see that $U \cup S$ is a minimum weight dominating set in G. Notice that every vertex in $V(G) - U$ has strictly positive weight. Moreover, it is not difficult to see that $G - U$ is strongly chordal if G is strongly chordal. Consider the following two problems:

Problem 1. Given a strongly chordal graph G with arbitrary real vertex weights, find a minimum weight dominating set in G.

Problem 2. Given a strongly chordal graph G with positive vertex weights and a set $S \subset V(G)$, find a minimum weight set of vertices of G which dominates S.

What we have shown above is that Problem 1 can be reduced to Problem 2. The algorithm which we will present in this section solves Problem 2.

For the remainder of this section we will assume that the graph G has positive vertex weights. Let $S \subset V(G)$ and, for each j, let $b_j = 1$ if $v_j \in S$ and $b_j = 0$ if $v_j \notin S$ (b is the characteristic function of S). Consider the following linear program:

$$
\begin{aligned}
P_2(G, S): \quad & \text{Minimize} && \sum_{i=1}^{n} w_i x_i, \\
& \text{subject to} && \sum_{i \sim j} x_i \geq b_j \quad \text{for each } j, \\
& && x_i \geq 0 \qquad \text{for each } i.
\end{aligned}
$$

There is a one-to-one correspondence between feasible 0–1 solutions to $P_2(G, S)$ and sets of vertices which dominate S. Moreover, an optimal 0–1 solution to $P_2(G, S)$ corresponds to a minimum weight set of vertices which dominates S. If G is strongly chordal, then the algorithm which will be presented solves $P_2(G, S)$ as well as the following dual:

$$
\begin{aligned}
D_2(G, S): \quad & \text{Maximize} && \sum_{j=1}^{n} b_j y_j, \\
& \text{subject to} && \sum_{j \sim i} y_j \leq w_i \quad \text{for each } i, \\
& && y_j \geq 0 \qquad \text{for each } j.
\end{aligned}
$$

It will follow from the algorithm that if G is strongly chordal, then $P_2(G, S)$ has

0-1 optimal solutions and if, in addition, G has integer vertex weights, then $D_2(G, S)$ has integer optimal solutions.

Note that the above linear programs are independent of the ordering of $V(G)$. Consequently, one might try to solve $P_2(G, S)$ for an arbitrary graph G in hopes of finding a minimum weight set of vertices which dominates S. Unfortunately, this approach can fail even for cycles and for chordal graphs which are not strongly chordal, as the following two examples demonstrate.

Example 1. Consider the 4-cycle, C_4, on the vertices v_1, v_2, v_3, v_4. Let $w_1 = w_2 = w_3 = w_4 = 1$ and let $S = \{v_1, v_2, v_3, v_4\}$. Then $x_1 = x_2 = x_3 = x_4 = \frac{1}{3}$ is a feasible solution to $P_2(C_4, S)$ of value $\frac{4}{3}$, and $y_1 = y_2 = y_3 = y_4 = \frac{1}{3}$ is a feasible solution to $D_2(C_4, S)$ of value $\frac{4}{3}$. By the duality theorem of linear programming, both of these solutions are optimal and hence neither $P_2(C_4, S)$ nor $D_2(C_4, S)$ has integer optimal solutions.

Example 2. Consider the graph G^* in Fig. 1. Let $w_i = 1$ for each i and let $S = V(G^*)$. Then $x_1 = x_2 = x_3 = 0$ and $x_4 = x_5 = x_6 = \frac{1}{2}$ is a feasible solution to $P_2(G^*, S)$ of value $\frac{3}{2}$, and $y_1 = y_2 = y_3 = \frac{1}{2}$ and $y_4 = y_5 = y_6 = 0$ is a feasible solution to $D_2(G^*, S)$ of value $\frac{3}{2}$. By the duality theorem of linear programming, both of these solutions are optimal and hence neither $P_2(G^*, S)$ nor $D_2(G^*, S)$ has integer optimal solutions.

Actually, it would have been surprising if solving $P_2(G, S)$ would locate a minimum weight set of vertices which dominates S for an arbitrary chordal graph G, since, as we have previously mentioned, the problem of locating a minimum cardinality dominating set in a chordal graph is NP-hard, whereas linear programs can be solved in polynomial time [16].

For the remainder of this section we will assume that G is strongly chordal and that $v_1, v_2, \ldots, v_n$ is a strong elimination ordering of G. To simplify the presentation of the algorithm we define a function h and a family of sets associated to $P_2(G, S)$ and its dual, $D_2(G, S)$: For each i,

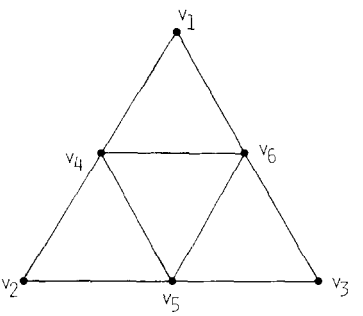


Fig. 1. G^*

$$h(i) = w_i - \sum_{j \sim i} y_j, \quad \text{and} \quad T_i = \{k\colon\ k \sim i \text{ and } y_k > 0\}.$$

Note that $h(i)$ is the slack in the dual constraint associated with vertex v_i, and T_i is the set of constraints in $P_2(G, S)$ containing x_i which must be at equality to satisfy the conditions of complementary slackness.

The algorithm has two stages. Stage one finds a feasible solution to $D_2(G, S)$ by scanning the vertices in the order $v_1, v_2, \ldots, v_n$, and stage two uses this solution to find a feasible 0-1 solution to $P_2(G, S)$ by scanning the vertices in the order $v_n, v_{n-1}, \ldots, v_1$. In this algorithm we will utilize a set T to assure that the complementary slackness conditions are satisfied. Initially, $T = \{1, 2, \ldots, n\}$, each $y_j = 0$, and each $x_i = 0$. If $b_j = 1$, then when v_j is scanned in stage one we add as much as possible to y_j without violating any dual constraints. Otherwise, y_j remains 0. In stage two, if $h(i) = 0$ and $T_i \subset T$ when v_i is scanned then we set $x_i \leftarrow 1$ and replace T by $T - T_i$. Otherwise x_i remains 0. A more formal description of the algorithm follows.

Algorithm I

Input: A strongly chordal graph G with strong elimination ordering $v_1, v_2, \ldots, v_n$ and positive vertex weights $w_1, w_2, \ldots, w_n$; and a subset S of vertices of G.

Output: Optimal solutions to $P_2(G, S)$ and $D_2(G, S)$.

Initially, $T = \{1, 2, \ldots, n\}$, each $y_j = 0$, and each $x_i = 0$.

Stage One: FOR $j = 1$ TO n DO
IF $b_j = 1$ THEN $y_j \leftarrow \min\{h(k)\colon\ k \sim j\}$.

Stage Two: FOR $i = n$ TO 1 BY -1 DO
IF $h(i) = 0$ and $T_i \subset T$ THEN DO
$x_i \leftarrow 1$
$T \leftarrow T - T_i$.

We will now establish the validity of Algorithm I.

Theorem 3.1. *Given as input a strongly chordal graph G with strong elimination ordering $v_1, v_2, \ldots, v_n$ and positive vertex weights $w_1, w_2, \ldots, w_n$, and a subset S of vertices of G, Algorithm* I *halts after $O(n + m)$ operations, and the final values of $x_1, x_2, \ldots, x_n$ and $y_1, y_2, \ldots, y_n$ are optimal solutions to $P_2(G, S)$ and $D_2(G, S)$ respectively.*

Proof. It is easy to see that the algorithm halts in $O(n + m)$ operations. In order to show that the algorithm finds optimal solutions to $P_2(G, S)$ and $D_2(G, S)$, it suf-

fices to show that these solutions are feasible and that they satisfy the conditions of complementary slackness.

(i) *Feasiblility of dual solution.* The instructions in stage one guarantee that $y_j \geq 0$ and $h(j) \geq 0$ for each j.

(ii) *Feasibility of primal solution.* Clearly, each x_i is either 0 or 1. Thus it suffices to show that if $b_j = 1$, then $x_i = 1$ for some $i \sim j$. Suppose that $b_j = 1$. By the choice of y_j, there is some $k \sim j$ such that $h(k) = 0$ and $\max T_k \leq j$. If $x_k = 1$ we are done. Otherwise, by the algorithm, T_k was not contained in T when v_k was scanned in stage two. Since, in stage two, the vertices are scanned in the order $v_n, v_{n-1}, \ldots, v_1$, there is some $l > k$ such that $x_l = 1$ and $T_l \cap T_k \neq \emptyset$. Let $i \in T_l \cap T_k$. Then $i \leq j$ since $\max T_k \leq j$. Thus $i \leq j$, $k < l$, and $l \sim i \sim k \sim j$, whence $l \sim j$ by Lemma 2.2, since $v_1, v_2, \ldots, v_n$ is a strong elimination ordering. Hence $l \sim j$ and $x_l = 1$. Consequently, the primal solution is feasible.

(iii) *Complementary slackness.* If $x_i > 0$, then $x_i = 1$ and so $h(i) = 0$, i.e., $\sum_{j \sim i} y_j = w_i$.

Suppose $y_j > 0$. Then $b_j = 1$. It is clear from the instructions that if $x_i = x_k = 1$, then $T_i \cap T_k = \emptyset$. Thus, $\sum_{i \sim j} x_i \leq 1$. Equality follows from the feasibility of the primal solution. □

Corollary 3.2. *If G is a strongly chordal graph with positive vertex weights and S is a set of vertices of G, then $P_2(G, S)$ has 0–1 optimal solutions. If, in addition, G has integer vertex weights then $D_2(G, S)$ has integer optimal solutions.*

This corollary is an immediate consequence of Theorem 3.1. It also follows from the fact that the matrix of the linear programs $P_2(G, S)$ and $D_2(G, S)$ is totally balanced (see Sections 5 and 6).

Corollary 3.3. *Under the same hypotheses as Theorem* 3.1, *Algorithm* I *locates a minimum weight set of vertices of G which dominates S.*

Let $w = (w_1, w_2, \ldots, w_n)$ be a vector of real numbers. We define $\gamma_w(G)$ to be the minimum weight of a dominating set in G weighted by w, and we define $\eta_w(G)$ to be the maximum number of vertices of G (with repetition allowed) such that each vertex v_i of G is equal or adjacent to at most w_i of them.

Corollary 3.4. *If G is a strongly chordal graph on n vertices then $\gamma_w(G) = \eta_w(G)$ for each n-vector w of non-negative integers.*

Proof. This is a restatement of Corollary 3.2 in the special case that $S = V(G)$. □

4. Independent domination

In this section we will present a linear algorithm to locate a minimum weight in-

dependent dominating set in a strongly chordal graph, given a strong elimination ordering.

Let G be a strongly chordal graph with strong elimination ordering $v_1, v_2, \ldots, v_n$ and arbitrary real vertex weights $w_1, w_2, \ldots, w_n$. For each j, let C_j be the subgraph of G induced by $\{v_i : i \lesssim j\}$. (Recall that $i \lesssim j$ means $i \sim j$ and $i \geq j$.) It follows from the definition of a strong elimination ordering that each C_j is complete. Moreover, each edge of G is in some C_j. Consequently, a set S of vertices of G is independent if and only if each C_j contains at most one vertex of S. Recall that S is dominating if and only if, for each j, S contains some v_i such that $i \sim j$. Consider the following linear program:

$$
\begin{aligned}
P_3(G): \quad & \text{Minimize} && \sum_{i=1}^{n} w_i x_i, \\
& \text{subject to} && \sum_{i \sim j} x_i \geq 1 \quad \text{for each } j, \\
& && \sum_{i \lesssim j} x_i \leq 1 \quad \text{for each } j, \\
& && x_i \geq 0 \quad \text{for each } i.
\end{aligned}
$$

It follows from the above comments that there is a one-to-one correspondence between independent dominating sets in G and feasible 0–1 solutions to $P_3(G)$. Moreover, an optimal 0–1 solution to $P_3(G)$ corresponds to a minimum weight independent dominating set in G. Notice that a set S of vertices of G is an independent dominating set in G if and only if it is a maximal independent set in G. Consequently, there exist independent dominating sets, and $P_3(G)$ is a feasible linear program. It will follow from the algorithm presented in this section that $P_3(G)$ has 0–1 optimal solutions, and that, if G has integer vertex weights, then the following dual program has integer optimal solutions:

$$
\begin{aligned}
D_3(G): \quad & \text{Maximize} && \sum_{j=1}^{n} (y_j - z_j), \\
& \text{subject to} && \sum_{j \sim i} y_j - \sum_{j \gtrsim i} z_j \leq w_i \quad \text{for each } i, \\
& && y_j, z_j \geq 0 \quad \text{for each } j.
\end{aligned}
$$

The algorithm that we present will solve the linear programs $P_3(G)$ and $D_3(G)$. To simplify the presentation of the algorithm we define a function f, and a family of sets: For each i,

$$f(i) = w_i + \sum_{j \gtrsim i} z_j - \sum_{j \sim i} y_j,$$

and

$$S_i = \{j: i \sim j \text{ and } y_j > 0\}.$$

Note that $f(i)$ is the slack in the dual constraint associated with vertex v_i, and S_i

is the set of "$\geq$" constraints in $P_3(G)$ containing x_i which must be at equality to satisfy the conditions of complementary slackness.

As in the preceding algorithm, this algorithm has two stages; stage one finds a feasible solution to $D_3(G)$ by scanning the vertices in the order $v_1, v_2, \ldots, v_n$; and stage two uses this solution to find a feasible 0–1 solution to $P_3(G)$ by scanning the vertices in the order $v_n, v_{n-1}, \ldots, v_1$. In this algorithm we will utilize a set S to assure that the conditions of complementary slackness are satisfied. Initially, $S = \{1, 2, \ldots, n\}$, each $y_j = 0$, each $z_j = 0$, and each $x_i = 2$. (The *interpretation* of $x_i = 2$ is that x_i has not yet been assigned as value.) If, at the time v_j is scanned in stage one, the dual constraint associated with v_j is violated, i.e., if $f(j) < 0$, then we add just enough to z_j to bring that constraint into feasibility. Otherwise, we add as much as possible to y_j without violating the dual constraint associated with v_j or with any previously scanned vertex. In stage two, if $x_i = 2$, $f(i) = 0$, and $S_i \subset S$ when v_i is scanned then we set $x_i \leftarrow 1$, $S \leftarrow S - S_i$, and $x_k \leftarrow 0$ for each v_k adjacent to v_i. A more formal description of the algorithm follows.

Algorithm II

Input: A strongly chordal graph G with strong elimination ordering $v_1, v_2, \ldots, v_n$, and real vertex weights $w_1, w_2, \ldots, w_n$.

Output: Optimal solutions to $P_3(G)$ and $D_3(G)$.

Initially, $S = \{1, 2, \ldots, n\}$, each $y_j = 0$, each $z_j = 0$, and each $x_i = 2$.

Stage One: FOR $j = 1$ TO n DO
If $f(j) < 0$ THEN $z_j \leftarrow -f(j)$
ELSE $y_j \leftarrow \min\{f(k): k \gtrsim j\}$.

Stage Two: For $i = n$ TO 1 BY -1 DO
IF $x_i = 2$, $f(i) = 0$, and $S_i \subset S$ THEN DO
$x_i \leftarrow 1$
$S \leftarrow S - S_i$
For each v_k adjacent to v_i DO
$x_k \leftarrow 0$.

We now establish the validity of Algorithm II.

Theorem 4.1. *Given as input a strongly chordal graph G with strong elimination ordering $v_1, v_2, \ldots, v_n$ and real vertex weights $w_1, w_2, \ldots, w_n$, Algorithm* II *halts after* $O(n+m)$ *operations and the final values of $x_1, x_2, \ldots, x_n$ and $y_1, y_2, \ldots, y_n, z_1, z_2, \ldots, z_n$ are optimal solutions to $P_3(G)$ and $D_3(G)$ respectively.*

Several lemmas are needed for the proof of this theorem.

Lemma 4.2. *For each j, $f(j) \geq 0$ at all times after scanning v_j in stage one.*

Proof. The fact that $f(j) \geq 0$ immediately after scanning v_j in stage one follows from the choice of z_j and y_j. The fact that $f(j) \geq 0$ after scanning each v_k where $k > j$ follows from the choice of y_k. □

Lemma 4.3. *For each j, $y_j \geq 0$ at the end of stage one.*

Proof. Given the choice of y_j, this is an immediate consequence of Lemma 4.2. □

Lemma 4.4. *Let $k \in \{1, 2, \ldots, n\}$. Suppose that $f(k) = 0$ immediately after scanning v_j in stage one, where $j \gtrsim k$. Then there is some $l \geq k$ such that $l \sim j$ and $x_l = 1$.*

Proof. We first claim that $f(k) = 0$ at the end of stage one and $S_k \cap \{j+1, j+2, \ldots, n\} = \emptyset$. After scanning v_j, the value of z_l remains fixed for each $l \leq j$. This fact, together with Lemma 4.2 and Lemma 4.3, guarantees that $f(k) = 0$ at all times after scanning v_j in stage one, and that $y_i = 0$ for each $i \sim k$ such that $i > j$. Thus $i \leq j$ for each $i \in S_k$.

Now, it is clear from the instructions that, for each l, if $x_l = 1$ at any time during the execution of the algorithm, then $x_l = 1$ at the end of the execution of the algorithm. If x_k was 2 and S_k was contained in S immediately prior to scanning v_k in stage two, then since $f(k) = 0$ at the end of stage one, x_k was assigned the value of 1 when v_k was scanned, and hence $x_k = 1$ at the end of stage two. Otherwise, either x_k was 0 or S_k was not contained in S. In the first case, $x_k = 0$ by virtue of the fact that $x_l = 1$ for some previously scanned neighbour v_l of v_k, i.e. for some $l \gtrsim k$. Now $k < l$, $k \leq j$, and $l \sim k \sim j$, whence $l \sim j$ since $v_1, v_2, \ldots, v_n$ is a strong elimination ordering. In the second case, there is some $l > k$ such that $x_l = 1$ and $S_l \cap S_k \neq \emptyset$. Let $i \in S_l \cap S_k$. Then $i \leq j$ since $i \in S_k$. Thus we have $i \leq j$, $k < l$, and $l \sim i \sim k \sim j$, and so $l \sim j$ by Lemma 2.2 and the fact that $v_1, v_2, \ldots, v_n$ is a strong elimination odering. □

Proof of Theorem 4.1. It is easy to see that the algorithm halts after $O(n+m)$ operations. In order to show that the algorithm finds optimal solutions to $P_3(G)$ and $D_3(G)$ it suffices to show that these solutions are feasible and that they satisfy the condition of complementary slackness.

(i) *Feasibility of dual solution.* Clearly $z_j \geq 0$ for each j. By Lemmas 4.2 and 4.3, $y_j \geq 0$ and $f(j) \geq 0$ for each j.

(ii) *Feasibility of primal solution.* We will first show that each x_j is either 0 or 1. Let $j \in \{1, 2, \ldots, n\}$. By the choice of z_j and y_j, there is some $k \gtrsim j$ such that $f(k)$ was 0 immediately after sanning v_j in stage one. By Lemma 4.4, there is some $l \geq k$ such that $l \sim j$ and $x_l = 1$. Thus, when x_l was assigned the value of 1, x_j was assigned

the value of either 1 or 0, depending upon whether j does or does not equal l. Hence, each x_j is either 0 or 1.

Now, by the algorithm, if $x_i = 1$ then $x_j = 0$ for each v_j adjacent to v_i, and if $x_i = 0$ then $x_j = 1$ for some v_j adjacent to v_i. Thus, the set $\{v_i : x_i = 1\}$ is an independent dominating set, and so the primal solution is feasible.

(iii) *Complementary slackness.* If $x_i > 0$, then $x_i = 1$ and so, by the algorithm, $f(i) = 0$, i.e. $\sum_{j \sim i} y_j - \sum_{j \gtrsim i} z_j = w_i$.

If $x_i = x_k = 1$ and $i \neq k$, then $S_i \cap S_k = \emptyset$, by the algorithm. Thus if $y_j > 0$, then $\sum_{i \sim j} x_i \leq 1$. Equality follows from the feasibility of the primal solution.

Suppose $z_j > 0$. Then $f(j)$ was 0 immediately after scanning v_j in stage one. Hence, by Lemma 4.4, there is some $l \lesssim j$ such that $x_l = 1$, whence $\sum_{i \lesssim j} x_i \geq 1$. Equality follows from the feasibility of the primal solution. □

Corollary 4.5. *If G is a strongly chordal graph with strong elimination ordering $v_1, v_2, \ldots, v_n$ and arbitrary real vertex weights, then $P_3(G)$ has 0–1 optimal solutions. If, in addition, the vertex weights are integral, then $D_3(G)$ has integer optimal solutions.*

Corollary 4.6. *Under the same hypotheses as Theorem* 4.1, *Algorithm* II *locates a minimum weight independent dominating set in G.*

Remark. Given as input a chordal graph G with perfect elimination ordering $v_1, v_2, \ldots, v_n$ and 0–1 vertex weights, Algorithm II will find optimal solutions to $P_3(G)$ and $D_3(G)$. Hence, Algorithm II can be used to locate minimum cardinality independent dominating sets in chordal graphs (cf. [11]).

5. The domatic number

The domatic number was defined and studied in [8]. A theoretical lower bound for the domatic number was established, and it was shown that the domatic number achieves this lower bound in trees, cliques and maximal outerplanar graphs.

Let $\delta(G)$ be the minimum degree of a vertex of the graph G.

Theorem 5.1 [8]. *For any graph G, $d(G) \leq \delta(G) + 1$.*

In this section we will show that the domatic number achieves this lower bound in strongly chodal graphs. We begin with some terminology.

A 0–1 matrix is *balanced* [2] if it does not contain as a submatrix an (edge-vertex) incidence matrix of an odd cycle, and is *totally balanced* [17] if it does not contain as a submatrix an incidence matrix of any cycle of length at least three. The *neighbourhood matrix* of a graph G on the vertices $v_1, v_2, \ldots, v_n$ is the $n \times n$ matrix whose (i, j)th entry is 1 if $i \sim j$ and is 0 otherwise. Thus the neighbourhood matrix

of G is the adjacency matrix of G plus the identity matrix. We have established the following relationship between strongly chordal graphs and totally balanced matrices.

Theorem 5.2 [12]. *A graph is strongly chordal if and only if its neighbourhood matrix is totally balanced.*

We can view a 0–1 matrix with no zero rows or columns as the incidence matrix of a hypergraph. A transversal in a hypergraph is a set of vertices meeting all of the edges. Berge proved the following duality theorem pertaining to transversals in hypergraphs:

Theorem 5.3 [2]. *If the incidence matrix of a hypergraph H is balanced, then the maximum number of pairwise disjoint transversals in H equals the minimum cardinality of an edge of H.*

By viewing the neighbourhood matrix of a graph as the incidence matrix of a hypergraph, we obtain the following corollary to Theorems 5.2 and 5.3.

Corollary 5.4. *If G is strongly chordal then $d(G) = \delta(G) + 1$.*

6. Totally balanced matrices

In [13], it was shown that if A is a balanced matrix with no zero rows and w is a vector (of appropriate dimension) of positive numbers, then the linear program

$$\begin{aligned} P_4(A, w)\text{:}\quad & \text{Minimize} && w \cdot x, \\ & \text{subject to} && Ax \geq 1, \\ & && x \geq 0 \end{aligned}$$

has 0–1 optimal solutions, and that if, in addition, w is integral, then the dual linear program

$$\begin{aligned} D_4(A, w)\text{:}\quad & \text{Maximize} && 1 \cdot y \\ & \text{subject to} && A^t y \leq w, \\ & && y \geq 0 \end{aligned}$$

has integer optimal solutions.

In this section we will show how to efficiently solve these linear programs in the case that A is totally balanced by reducing $P_4(A, w)$ to the problem of locating a minimum weight set of vertices which dominates some subset of vertices of a strongly chordal graph.

For a given matrix A, let A^* be the matrix

$$\begin{bmatrix} I & A \\ A^t & J \end{bmatrix}$$

where J is the matrix (of appropriate dimension) of all 1's. The following theorem leads to another relationship between strongly chordal graphs and totally balanced matrices. The proof is simple and is omitted.

Theorem 6.1. *A is a totally balanced matrix if and only if A^* is a totally balanced matrix.*

Corollary 6.2. *Let A be a totally balanced matrix. Then A^* is the neighbourhood matrix of a strongly chordal graph.*

Proof. A^* is the neighbourhood matrix of a graph since it is symmetric and has 1's on the main diagonal. The associated graph is strongly chordal by Theorems 5.2 and 6.1. □

Now, let A be an $m \times n$ totally balanced matrix with no zero rows and let w be an n-vector of positive numbers. Let M be an m-vector all of whose entries are greater than $1 \cdot w$. Consider the following linear program:

$$\begin{array}{lll} P_5(A, w): & \text{Minimize} & M \cdot y + w \cdot x, \\ & \text{subject to} & Iy + Ax \geq 1, \\ & & A^t y + Jx \geq 0, \\ & & y \geq 0, \\ & & x \geq 0. \end{array}$$

It is easy to see that, in any optimal solution to $P_5(A, w)$, $y = 0$, and thus there is a one-to-one correspondence between optimal solutions to $P_4(A, w)$ and $P_5(A, w)$. Consequently, we can solve $P_4(A, w)$ by solving $P_5(A, w)$. Moreover, $P_5(A, w)$ is precisely the linear program associated with locating a minimum weight dominating set of a subset of vertices of the strongly chordal graph G associated with A^*. Thus, given a strong elimination ordering of G, we can use Algorithm I to solve $P_5(A, w)$. (As we have previously mentioned, this ordering can be found in polynomial time.) Note that the first stage of Algorithm I will also solve $D_4(A, w)$.

We remark that, with simple modifications, Algorithm I can be used to solve $P_4(A, w)$ and $D_4(A, w)$ directly, without solving $P_5(A, w)$, as long as the rows and columns of A are permuted to correspond to a strong elimination ordering of the graph associated with A^*.

Hoffman, Kolen and Sakarovitch [15] have independently obtained similar results. They say that a 0–1 matrix A is in *standard greedy form* if its rows and columns have been permuted in a way that corresponds exactly to a strong elimination ordering of the graph associated with A^*. They present polynomial algorithms

to put a totally balanced matrix into standard greedy form (cf. [1]) and to solve linear programs of the form

$$\begin{array}{lll} \text{Minimize} & w \cdot x, & \text{and} \quad \text{Maximize} \quad b \cdot y, \\ \text{subject to} & Ax \geq b, & \text{subject to} \quad A^{t} y \leq w, \\ & x \geq 0 & \qquad\qquad\quad y \geq 0, \end{array}$$

where A is in standard greedy form, w is a vector of positive numbers, and $b^{t} = [b_1, b_2, \ldots, b_n]$ satisfies $b_1 \geq b_2 \geq \cdots \geq b_n \geq 0$. The method used in the latter algorithm is essentially the same as that described in this paper.

Acknowledgement

The author wishes to thank Pavol Hell for his advice and encouragement during the writing of the Ph.D. thesis which led to this paper.

References

[1] R.P. Anstee and M. Farber, Characterizations of totally balanced matrices, J. Algorithms, to appear.

[2] C. Berge, Balanced matrices, Math. Programming 2 (1972) 19–31.

[3] T. Beyer, A. Proskurowski, S. Hedetniemi and S. Michell, Independent domination in trees, Proc. Eighth Southeastern Conf. Combinatorics, Graph Theory, and Computing (Utilitas Math. Winnipeg, 1977) 321–328.

[4] K.S. Booth and J.H. Johnson, Dominating sets in chordal graphs, SIAM J. Comput. 11 (1982) 191–199.

[5] G.J. Chang, Private communication, 1982.

[6] G.J. Chang and G.L. Nemhauser, The k-domination and k-stability problems on graphs, T.R. 540, School of OR/IE, Cornell University, 1982.

[7] E. Cockayne, S. Goodman, and S. Hedetniemi, A linear algorithm for the domination number of a tree, Inform. Process. Letters 4 (1975) 41–44.

[8] E.J. Cockayne and S.T. Hedetniemi, Towards a theory of domination in graphs, Networks 7 (1977) 247–261.

[9] M. Farber, Domination and duality in weighted trees, Proc. Twelfth Southeastern Conf. Combinatorics, Graph Theory, and Computing (Utilitas Math., Winnipeg, 1982).

[10] M. Farber, Applications of l.p. duality to problems involving independence and domination, Ph.D. Thesis, Rutgers University, New Brunswick, NJ (January 1982); also issued as Technical Report 81–13, Computing Science Department, Simon Fraser University, 1981.

[11] M. Farber, Independent domination in chordal graphs, Operations Research Letters 1 (1982) 134–138.

[12] M. Farber, Characterizations of strongly chordal graphs, Discrete Math. 43 (1983) 173–189.

[13] D.R. Fulkerson, A.J. Hoffman and R. Oppenheim, On balanced matrices, Math. Programming Study 1 (1974) 120–132.

[14] M.R. Garey and D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness (Freeman, San Francisco, CA, 1979).

[15] A.J. Hoffman, A.W.J. Kolen and M. Sakarovitch, Totally-balanced and greedy matrices, submitted to SIAM J. Algebraic Discrete Methods.

[16] L.G. Khachian, A polynomial algorithm in linear programming, Soviet Math. Dokl. 20 (1979) 191–194.
[17] L. Lovász, Combinatorial Problems and Exercises (North-Holland, Amsterdam, 1979).
[18] A. Lubiw, Γ-free matrices, Masters thesis, University of Waterloo, Waterloo, Ont. (October 1982).
[19] K.S. Natarajan and L.J. White, Optimum domination in weighted trees, Inform. Process. Letters 7 (1978) 261–265.
[20] D.J. Rose, Triangulated graphs and the elimination process, J. Math. Anal. Appl. 32 (1970) 597–609.
[21] D.J. Rose, R.E. Tarjan and G.S. Lueker, Algorithmic aspects of vertex elimination on graphs, SIAM J. Comput. 5 (1976) 266–283.
[22] D.M. Simmons, Linear Programming for Operations Research (Holden-Day, San Francisco, CA, 1972).
[23] P.J. Slater, R-domination in graphs, J. Assoc. Comput. Mach. 23 (1976) 446–450.